

CA One Report

Student ID: 20065898

Module: Programming (C# and Python)

Date: 08 April 2026

1. Overview

This assignment was completed in four parts as requested, with C# used for Section I (Que.1 and Que.2) and Python used for Section II (Que.3 and Que.4). The workspace was organized under the student-id folder (20065898) with separate subfolders for each question. The implementation focuses on object-oriented design, appropriate data structures, networking, persistence, and web scraping.

2. Que.1 (C#): Churros Food Truck Queue System

A menu-driven console application was implemented to simulate stall-owner operations for churros ordering and delivery.

Key features implemented:

- Displays churros menu with prices.
- Supports Place order, Deliver order, and Exit options.
- Generates incremental order numbers.
- Calculates payment before queue insertion.
- Uses FIFO order handling through Queue<Order>.

Object-oriented design:

- Churros class includes item name and price, with constructor and properties.
- Order class includes order number, order details, quantity, and bill.
- Order methods include placing an order, paying bill, and collecting order.
- Encapsulation is maintained with private state (e.g., payment status) and controlled property access.

Testing:

- Unit test project (ChurrosApp.Tests) verifies PayBill() returns correct total and updates Bill property.
- Test result: passed.

3. Que.2 (C#): Periodic Table Elements (1-30)

A C# console program was created to store and retrieve the first 30 periodic table elements by atomic number.

Implementation details:

- Data structure used: Dictionary<int, Element>.
- Element record stores atomic number, element name, class, and short information.
- User provides atomic number input.
- Program validates input and displays details when present.
- Loop continues with y and terminates on n.

This satisfies the requirement to provide element information via atomic number using a suitable structure.

4. Que.3 (Python): EasyDrive TCP Client-Server Registration

A connection-oriented (TCP) client-server system was implemented.

Client responsibilities:

- Collect customer name, address, PPS number, and driving license data.
- License can be submitted as file content (base64) or text input.
- Sends registration request as JSON to server.

Server responsibilities:

- Accepts TCP connections.
- Validates request payload.
- Stores registration in SQLite (disk-persistent relational database).

- Generates and returns unique registration number (ED-YYYYMMDD-#####).

Persistence:

- Database file: Que.3/easydrive.db.
- Table stores customer data, license data, timestamps, and registration numbers.

Verification:

- Successful connection and registration response were tested.
- Database query confirmed persisted records.

File naming note:

- Existing files Que4_server.py and Que4_client.py were retained.
- Added Que3_server.py and Que3_client.py entry points for clearer question mapping.

5. Que.4 (Python): Travel Books Web Scraper

A Python scraper was developed for:

https://books.toscrape.com/catalogue/category/books/travel_2/index.html

Implementation:

- Uses requests to fetch HTML.
- Uses BeautifulSoup to parse book entries.
- Extracts book title, rating, and price.
- Saves output to CSV (travel_books.csv).
- Reads all rows from CSV and prints to terminal at program end.

Verification:

- Dependencies installed (requests, beautifulsoup4).
- Script executed successfully and printed all rows from CSV.

6. Folder Organization and Submission Notes

The work is organized into separate folders (Que.1, Que.2, Que.3, Que.4) under the student-id main folder. This aligns with structural requirements.

Before final Moodle submission, the following must be attached manually:

- Screenshots of program outputs for Que.1 and Que.2.
- Screenshots of client-server success and database state for Que.3.
- Screenshot of Que.4 execution and CSV output.
- Electronic Assignment Cover Sheet.
- GitHub repository link: <https://github.com/syedhadihussain5125/20065898>

7. External Help and Resources Attributed

- 1. Microsoft Learn - .NET, C#, and MSTest documentation.**
- 2. Python official documentation (socket, sqlite3, csv, json, base64).**
- 3. Requests documentation for HTTP retrieval.**
- 4. Beautiful Soup documentation for HTML parsing.**
- 5. Wikipedia periodic table reference for element data context.**
- 6. PubChem element pages for concise element information.**